UCS503P - Project

MediBridge

Healthcare Access and Financial Assistance Platform

Submitted by:

1024160137 Kunal Motwani
1024160058 Suryansh Pahuja

Group: MediBridge, BE Third Year, CSE

Submitted to:

Dr. Sandeep Kaur

Mid-Semester Evaluation
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

List of Figures ii

1 Introduction 1 3

1.2 Project Objectives 3

2 Methodology and Design 3

2.1 System Architecture 3

2.1.1 High-Level Design 3

2.1.2 Technical Stack 3
2.1.3 Use Case Model 4
2.1.4 User Workflow (Activity Diagram) 4
2.1.5 Data Flow Diagrams 4
2.1.6 Entity-Relationship Model 7
2.2 Implementation Details 7
2.2.1 Authentication and Health ID Module 7
2.2.2 Role-Based Access and Protected Routes 9
2.2.3 Patient Profile and Emergency Profile 10
2.2.4 Hospital and Doctor Discovery 10
2.2.5 Appointment Module 10
2.2.6 Medical Records Module 11
2.2.7 Doctor Access and Consent Module 11
2.2.8 Prescription Module 12
2.2.9 Financial Assistance Module 13
2.2.10 Fundraising Module 13
2.2.11 Emergency and NFC-Oriented Workflow 14
2.3 Current Implementation Status 14
3 Results and Evaluation 16
3.1 Testing Methodology 16
3.1.1 Functional Testing 16
3.1.2 Security-Oriented Testing 16
3.1.3 Integration Testing 16
3.2 Performance Metrics 17
3.3 Analysis 17
3.3.1 Objectives Achieved 17
3.3.2 Challenges Encountered 17
3.3.3 Current Limitations 18
4 Conclusion 19
4.1 Summary of Work 19
4.2 Key Findings 19
4.3 Future Work and Recommendations 20
4.4 Final Remarks 20

1.1 Background and Context . . . . . 1 1.1.1 Problem Statement 1

List of Figures

2.1 Use case diagram of the MediBridge platform 5

2.2 Activity diagram of the patient workflow 6
2.3 DFD Level 0 – context diagram 7
2.4 DFD Level 1 – overall MediBridge system 8
2.5 Entity-relationship diagram of the core MediBridge entities 9
2.6 DFD Level 2 – Medical Records and Doctor Access (Process 3.0) 12
2.7 DFD Level 2 – Financial Assistance and ML Prediction (Process 4.0) 15

Chapter 1 Introduction

1.1 Background and Context

Healthcare information is often fragmented across hospitals, doctors, medical documents, and financial service providers. Patients may have difficulty discovering suitable healthcare facilities, maintaining a continuous record of their medical history, understanding treatment costs, and arranging financial assistance when treatment becomes expensive.

MediBridge was proposed as a centralized, patient-focused platform that combines healthcare discovery, medical information management, and financial assistance into a unified web application. The original proposal identifies the core engineering objective as improving healthcare accessibility, reducing delays, and supporting better decision-making by bringing these workflows together.

The proposed system follows an MVP-first development approach. The initial focus was placed on patient registration, healthcare profiles, hospital discovery, medical record management, and progressively extending the platform towards doctor interaction, financial assistance, and emergency access.

1.1.1 Problem Statement

Patients commonly encounter the following difficulties:

Healthcare information about hospitals, doctors, treatment options, and costs is distributed across multiple sources.

Medical records may be maintained separately by different hospitals, making continuity of care difficult.

Patients may have limited visibility into who has accessed their medical information.

Treatment costs can create a financial barrier even when suitable healthcare services are available.

In emergencies, critical information such as blood group, allergies, medications, and emergency contacts may not be immediately accessible.

The project therefore aims to provide a single platform through which a patient can manage health information, discover hospitals and doctors, interact with healthcare providers, and access financial and emergency support.

1.2 Project Objectives

The major objectives of the project, derived from the original proposal and refined according to the implementation completed by the mid-semester stage, are:

1. **Develop a secure healthcare identity and authentication system:** provide patient and doctor registration, login, role-based access, password hashing, token-based authentication, and automatic generation of a unique MediBridge Health ID for patients.
2. **Develop centralized medical information management:** allow patients to maintain profiles, store medical records, and access structured prescriptions while allowing authorized doctors to interact with patient information.
3. **Implement healthcare discovery and appointment workflows:** provide hospital search, hospital details, doctor information, filtering, and appointment-request functionality.
4. **Provide privacy-aware doctor access:** implement patient-controlled access through a PINbased mechanism, temporary access sessions, access logs, and session revocation.
5. **Integrate financial assistance workflows:** implement loan application storage, rule-based eligibility evaluation, and an external ML-model API interface for loan prediction.
6. **Provide an emergency health-access mechanism:** store a limited emergency profile and expose an emergency-facing route that can be associated with a patient's Health ID and NFC-based workflow.

Chapter 2

Methodology and Design

2.1 System Architecture

The implemented MediBridge system follows the three-tier architecture proposed in the project proposal, consisting of a React frontend, Node.js/Express backend, and MongoDB-based persistence layer. An additional Python Flask service is used for the financial prediction component.

2.1.1 High-Level Design

The system is organized into the following major components:

1. **Frontend:** a React single-page application built with Vite. The frontend provides separate patient and doctor workflows, protected routes, role-based navigation, profile management, healthcare discovery, record management, appointments, and financial application interfaces.
2. **Backend API:** a Node.js and Express.js REST API that provides authentication, profile management, hospital search, medical-record operations, prescriptions, doctor access, financial services, fundraisers, emergency profiles, and appointment operations.
3. **Database:** MongoDB accessed through Mongoose. The database uses dedicated models for users, hospitals, doctors, medical records, prescriptions, access sessions, access logs, emergency profiles, loan applications, fundraisers, donations, and appointments.
4. **ML Service:** a separate Python Flask application exposes a /predict endpoint for loan prediction. The main backend communicates with this service through HTTP.
5. **Cloud file storage:** uploaded profile and medical documents are integrated with Cloudinary through the backend.

The overall design therefore preserves the proposal's separation between presentation, business logic, and persistence while adding a separate service boundary for the financial prediction component.

2.1.2 Technical Stack

Frontend: React 19, Vite, React Router, Zustand

Styling/UI: Tailwind CSS and custom CSS **Backend:**

Node.js and Express.js

Database: MongoDB with Mongoose ODM

Authentication: JWT, refresh tokens, bcryptjs

Validation: Joi

Security middleware: Helmet and express-rate-limit

File uploads: Multer and Cloudinary

Frontend utilities: Axios, React Hot Toast, Lucide React, QRCode React

ML service: Python, Flask, Flask-CORS

Development/build tools: Vite, npm and Oxlint

2.1.3 Use Case Model

The functional scope of MediBridge is summarised by the use case diagram in Figure 2.1. The system has three actors.

The **Patient** registers and receives a Health ID, manages the emergency profile and the fourdigit access PIN, uploads and views medical records, views the access log and revokes doctor access, searches hospitals and doctors, books hospital appointments, applies for a medical loan, and launches a crowdfunding campaign to cover a funding gap.

The **Doctor** searches for a patient by Health ID, verifies the patient's PIN to obtain 24-hour access, and then views the patient's clinical history, adds clinical records, notes and vitals, and writes digital prescriptions. Doctors also manage the appointments of their hospital.

Emergency personnel (first responders) scan an NFC card to view the limited emergency information of a patient without needing an account.

Login is included by the authenticated use cases, and password reset extends Login. The diagram represents the intended functional scope of the platform; the implementation status of each function is summarised in Section 2.3 (for example, NFC reading and writing remains future work).

2.1.4 User Workflow (Activity Diagram)

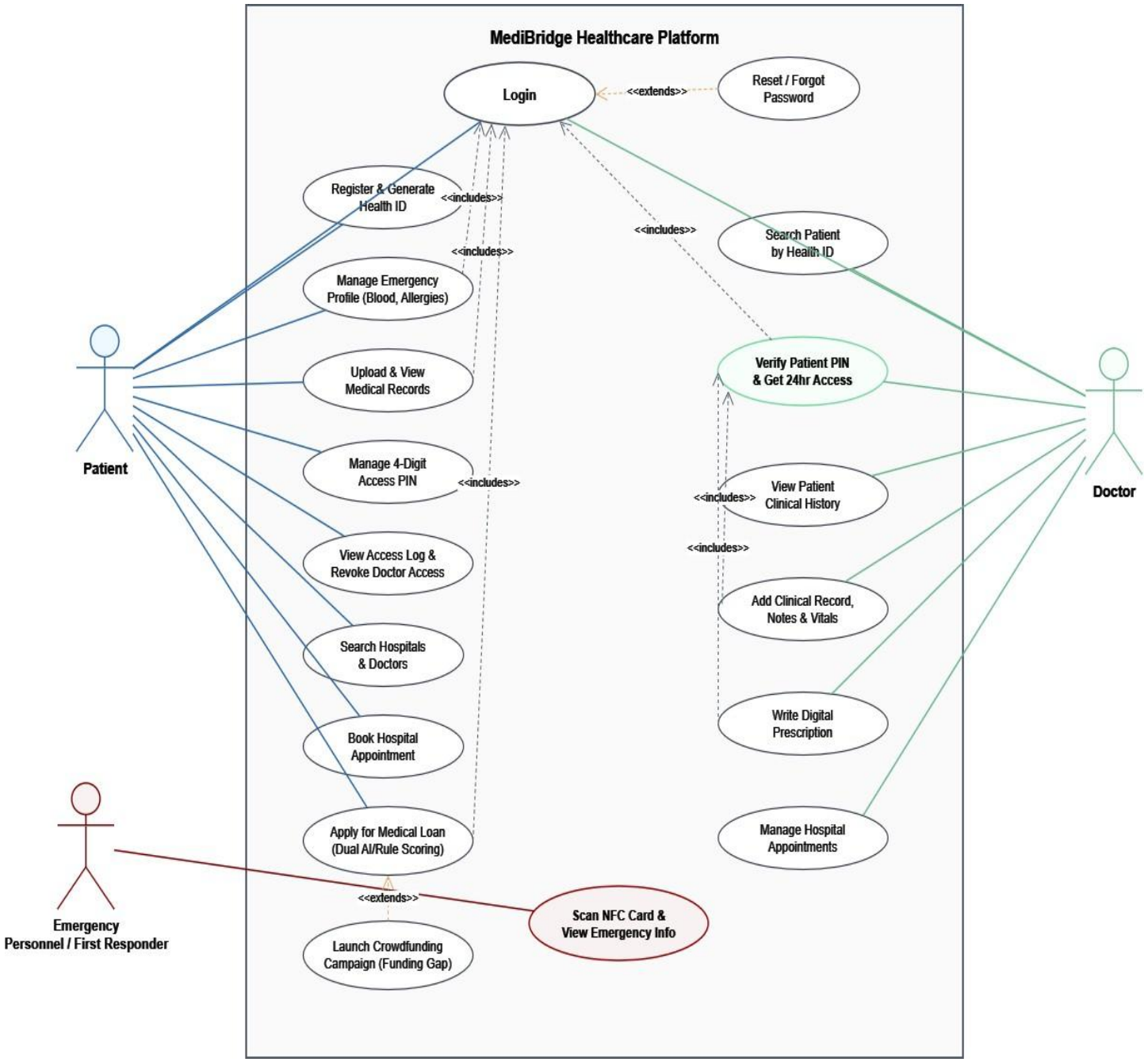
Figure 2.2 shows the end-to-end workflow of a patient using the platform. A new user registers and receives a Health ID; a registered user logs in directly. If the credentials are not valid, the user returns to the login step. After successful login the user reaches the dashboard and selects a service: profile management, hospital search followed by appointment booking, medical records (view or upload), financial assistance (eligibility check), fundraising (create a campaign or donate), or the emergency profile. Every branch ends with logout.

2.1.5 Data Flow Diagrams

Data movement through the system is modelled with Data Flow Diagrams (DFDs) at three levels. The notation follows standard DFD conventions: rectangles are external entities, circles are processes, open-ended bars are data stores, and labelled arrows are data flows. External entities never connect directly to data stores; all data passes through a process.

Level 0 (context diagram). Figure 2.3 treats the whole application as a single process, *0. MediBridge*. The Patient sends data and requests and receives healthcare services and results. The Doctor sends patient care data and receives authorized patient records. The external ML Prediction Service receives loan application data and returns a prediction result.

Level 1 (overall system). Figure 2.4 decomposes the system into five processes: *1.0 Authentication and Profile*, *2.0 Hospital and Appointments*, *3.0 Medical Records and Doctor Access*, *4.0 Financial Assistance* and *5.0 Fundraising*. They use nine logical data stores: D1 User Database, D2 Hospital and Doctor Database, D3 Appointment Database, D4 Medical Records, D5 Access Sessions and Logs, D6 Prescriptions, D7 Loan Applications, D8 Fundraisers and Donations, and D9 Emergency Profiles. These stores are logical groupings for readability and do not correspond to nine separate physical databases; in


MediBridge Healthcare Platform
Login
Reset / Forgot Password
<<extends>>
Register & Generate Health ID
<<includes>>
Manage Emergency Profile (Blood, Allergies)
<<includes>>
Upload & View Medical Records
Manage 4-Digit Access PIN
<<includes>>
View Access Log & Revoke Doctor Access
Search Hospitals & Doctors
Book Hospital Appointment
Apply for Medical Loan (Dual AI/Rule Scoring)
<<extends>>
Launch Crowdfunding Campaign (Funding Gap)
Patient
Emergency Personnel / First Responder
Scan NFC Card & View Emergency Info
<<includes>>
Search Patient by Health ID
Verify Patient PIN & Get 24hr Access
<<includes>>
View Patient Clinical History
<<includes>>
Add Clinical Record, Notes & Vitals
Write Digital Prescription
Manage Hospital Appointments
Doctor

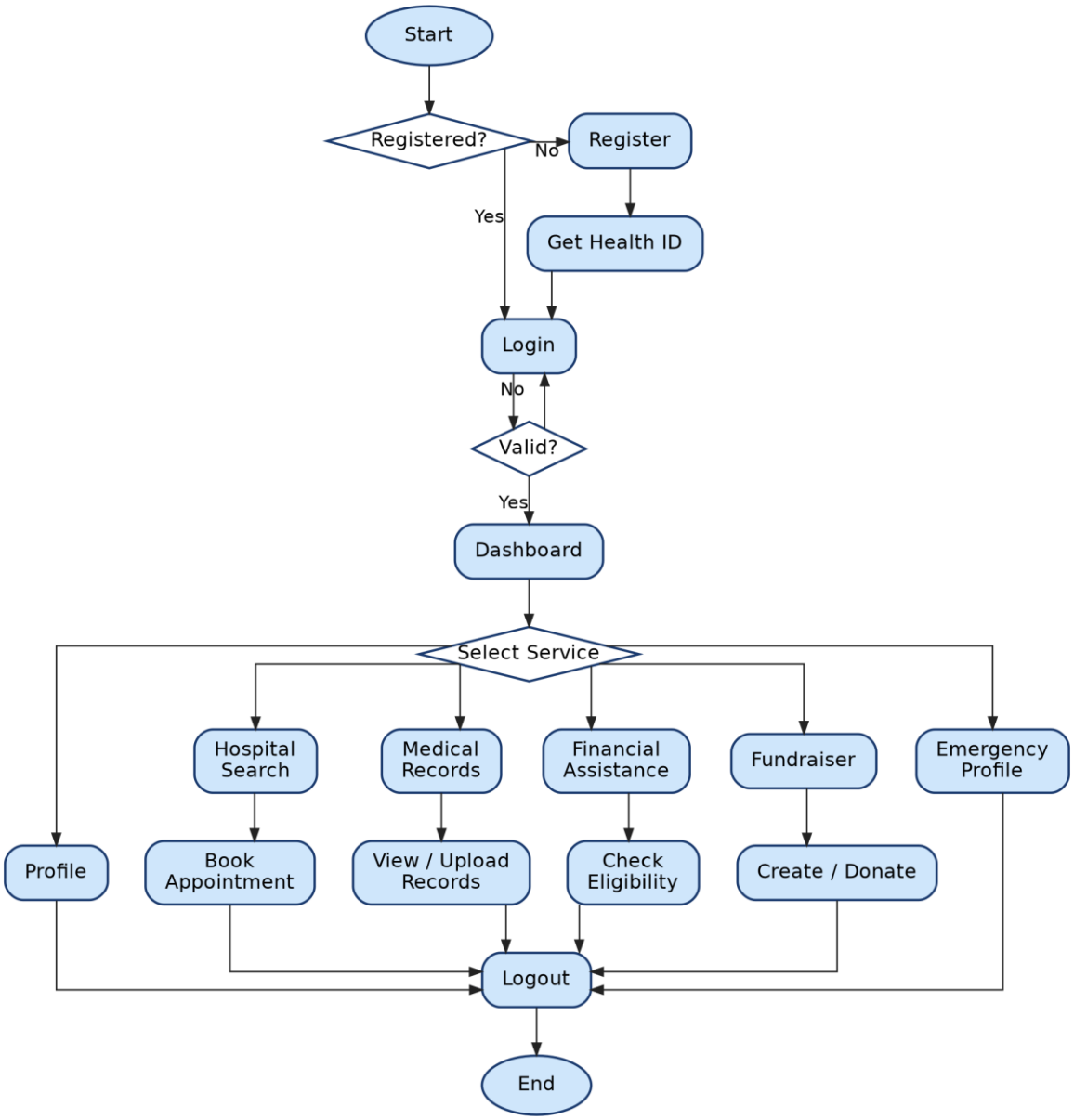


Figure 2.2: Activity diagram of the patient workflow

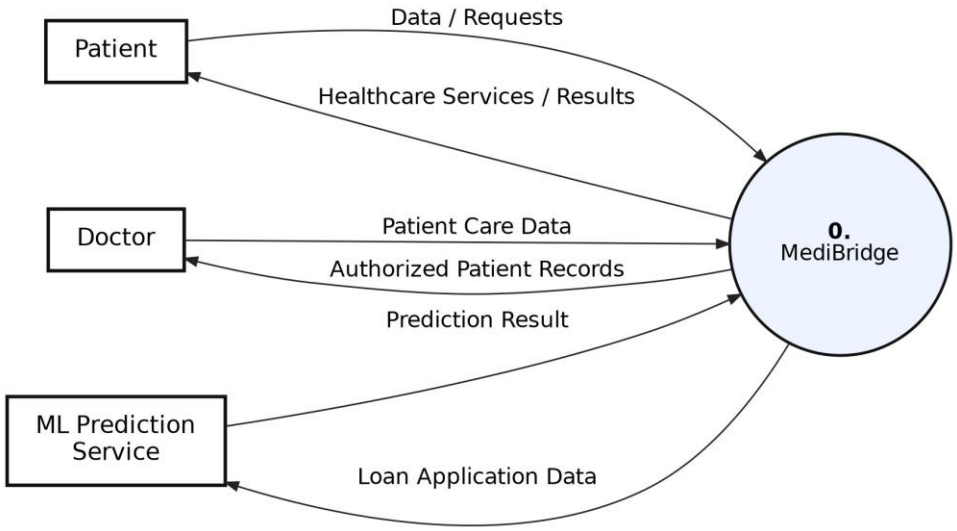


Figure 2.3: DFD Level 0 – context diagram

the implementation they are MongoDB collections. Loan data reaches the ML Prediction Service only through Process 4.0. The external flows are the same as in Level 0, so the two levels are balanced.

Level 2. The two most complex processes are decomposed further: Process 3.0 in Section 2.2.7 (Figure 2.6) and Process 4.0 in Section 2.2.9 (Figure 2.7).

2.1.6 Entity-Relationship Model

Figure 2.5 shows the core entities of the persistence layer and their relationships. The **User** entity (patients and doctors, identified by role) holds the Health ID and the hashed access PIN. A user has one **Emergency-Profile** (a weak entity that depends on the user), owns many **Medical-Records**, and applies for many **Loan-Applications**. A user grants many **Access-Sessions**, each received by one **Doctor** and carrying permissions, status and an expiry time. A doctor works in one **Hospital**, updates medical records, and prescribes **Prescriptions**, each of which is also associated with the patient it is issued to.

The diagram shows the core entities only. Appointments, fundraisers, donations and access logs, which are also part of the implemented database, are omitted for readability.

2.2 Implementation Details

2.2.1 Authentication and Health ID Module

The authentication module was implemented for both patients and doctors. Patient registration stores the user's personal information and access PIN, whereas doctor registration additionally stores specialization, qualification, license number, experience, and optional hospital association.

Passwords and patient access PINs are stored using bcrypt hashing. Authentication uses JWT-based access tokens and refresh tokens. The backend also applies rate limiting to authentication endpoints.

Each patient automatically receives a unique Health ID in the form MB-YYYY-XXXXXX during user

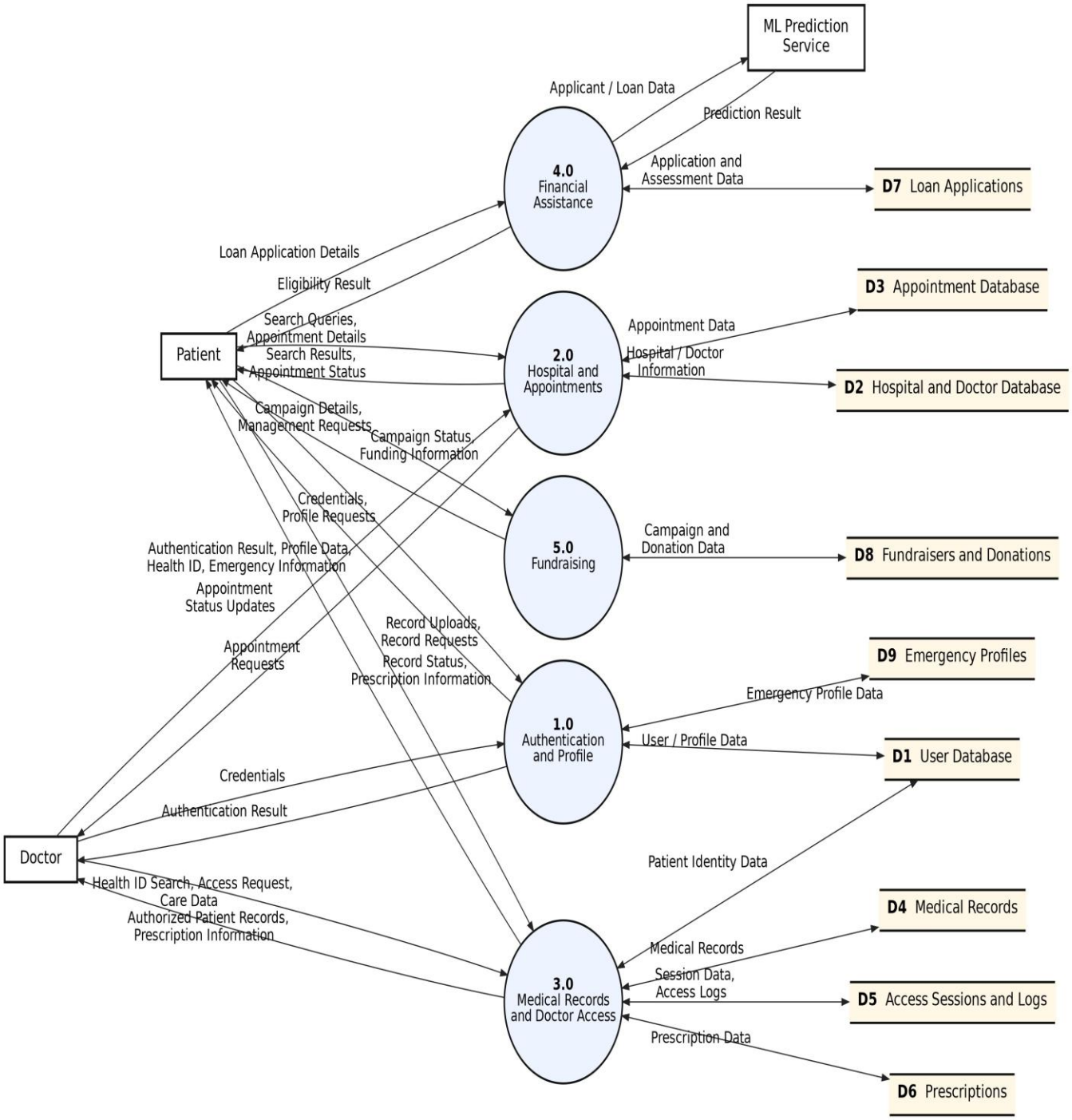


Figure2.4:DFDLevel1–overallMediBridgesystem

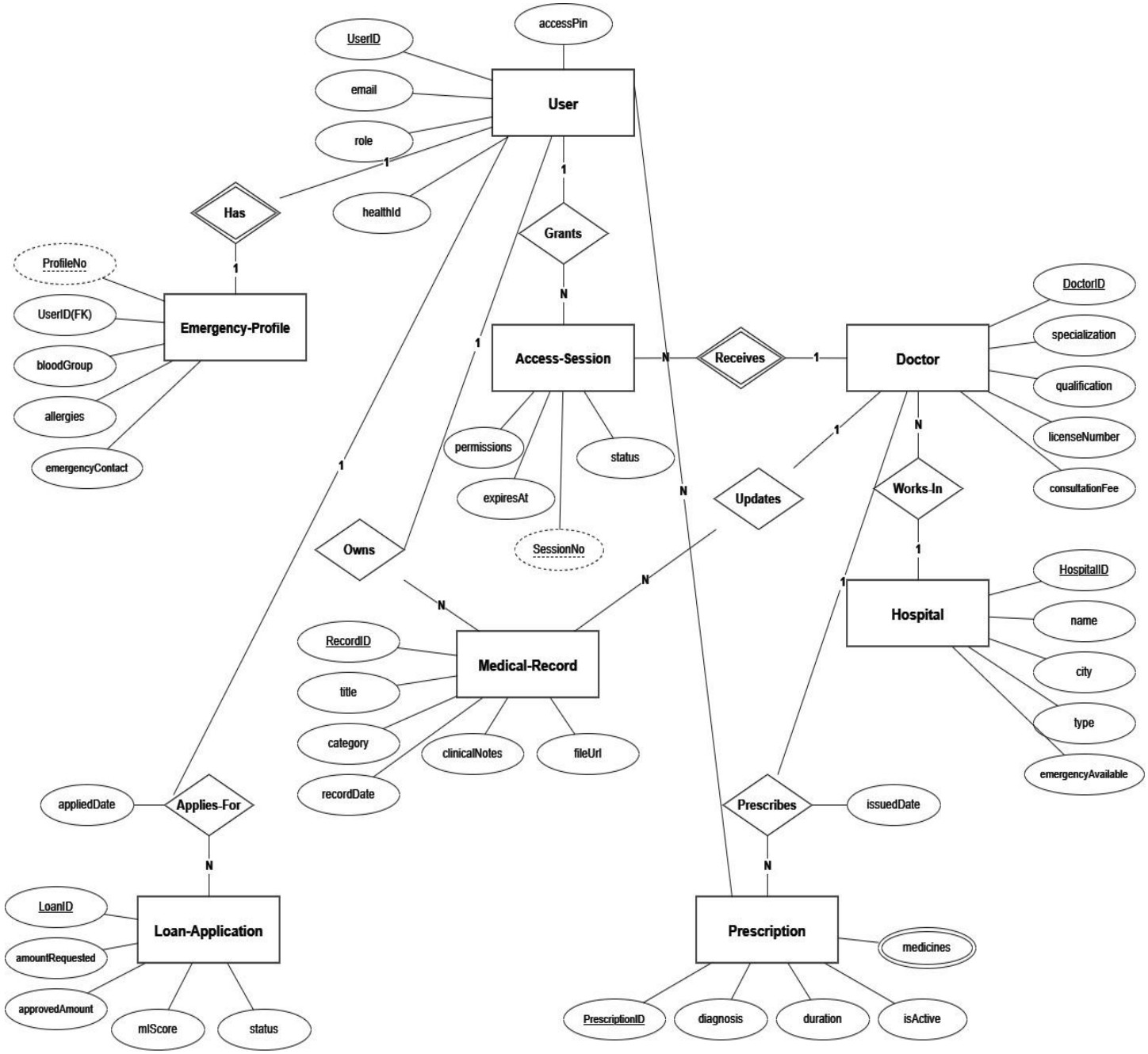


Figure2.5:Entity-relationshipdiagramofthecore MediBridgeentities

creation. The Health ID is used throughout the platform to identify patients and supports the doctoraccess and emergency-access workflows.

2.2.2 Role-Based Access and Protected Routes

The frontend contains dedicated route guards for authenticated users and for role-based access. Patient and doctor pages are therefore separated into protected workflows.

The backend similarly associates users with roles such as patient, doctor, and admin. This provides a foundation for limiting access to sensitive healthcare operations.

2.2.3 Patient Profile and Emergency Profile

The profile module supports retrieval and modification of patient information including name, phone number, date of birth, gender, and address. Profile image upload is supported through Cloudinary.

A separate emergency profile is maintained for information such as:

Blood group
Allergies
Chronic conditions
Current medications
Emergency contacts
Insurance information Organ-donor
status
Additional emergency notes

The emergency profile is intentionally separated from the full medical record so that an emergency-access workflow can expose a restricted subset of information.

2.2.4 Hospital and Doctor Discovery

The healthcare discovery module implements hospital search with support for:

Search by hospital name and related text fields
City and state filtering
Hospital type filtering
Specialization filtering
Emergency-availability filtering
Minimum-rating filtering
Pagination and sorting

Hospital records contain specializations, address information, ratings, bed information, emergency availability, insurance information, and verification state. Hospital details additionally expose associated doctors. The doctor model stores specialization, qualifications, experience, consultation fee, availability, rating, and hospital association.

2.2.5 Appointment Module

The implementation extends the initial proposal with an appointment workflow. Patients can submit appointments against a hospital by providing:

Preferred date

Specialist
Preferred time period

Appointments initially enter a pending state. Doctor-side endpoints support viewing appointments associated with the doctor's hospital and updating appointment status to booked, rejected, or completed.

2.2.6 Medical Records Module

The medical-record subsystem provides centralized record storage for patients. Each record may contain:

Title and category
Description and clinical notes
Diagnosis
Vital signs
Hospital and doctor information Record date
Tags
Uploaded document metadata

Records may be uploaded by the patient, by a doctor, or imported through the implemented mock hospital-fetch workflow. Medical documents can be uploaded through Multer and stored using Cloudinary. The backend provides record retrieval, filtering, pagination, update, and deletion operations.

2.2.7 Doctor Access and Consent Module

A major implementation of the proposal's privacy objective is the patient-controlled doctor-access mechanism. A doctor first searches for the patient using the Health ID. The patient then authorizes access using the four-digit access PIN.

On successful verification:

A doctor-patient access session is created.
The session is valid for 24 hours.
The access event is recorded in an audit log.
The session contains permissions for records, prescriptions, emergency information, and creation of medical information.

Patients can inspect their active access sessions and revoke a session before expiry. Access-log records therefore provide a history of access grants and revocations. The database also defines an expiry index on access sessions, allowing temporary access to be represented as time-bounded rather than permanent authorization.

The data flow of this module is shown in Figure 2.6, which decomposes Process 3.0 of the Level 1 DFD. A doctor first submits a Health ID to *3.1 Find Patient by Health ID*, which retrieves the patient's identity data from the User Database (D1) and returns the search result; no medical data is revealed at this stage. The doctor then submits the access PIN to *3.2 Verify Access PIN*, which checks it against the stored (hashed) PIN data. On successful verification, *3.3 Create or Validate Access Session* creates a temporary session or validates an existing one using the Access Sessions and Logs store (D5); expiry and revocation are reflected in this session data.

Protected data is released only after authorization. *3.4 View or Update Medical Records* and *3.5 Retrieve or Manage Prescriptions* act on the Medical Records (D4) and Prescriptions (D6) stores and receive session authorization from 3.3 before returning data to the doctor. The doctor never accesses D4

or D6 directly. Patients use the same processes to upload records and to view their own records and prescriptions. Finally, *3.6 Record Access Activity* writes every access event to D5 as the audit log.

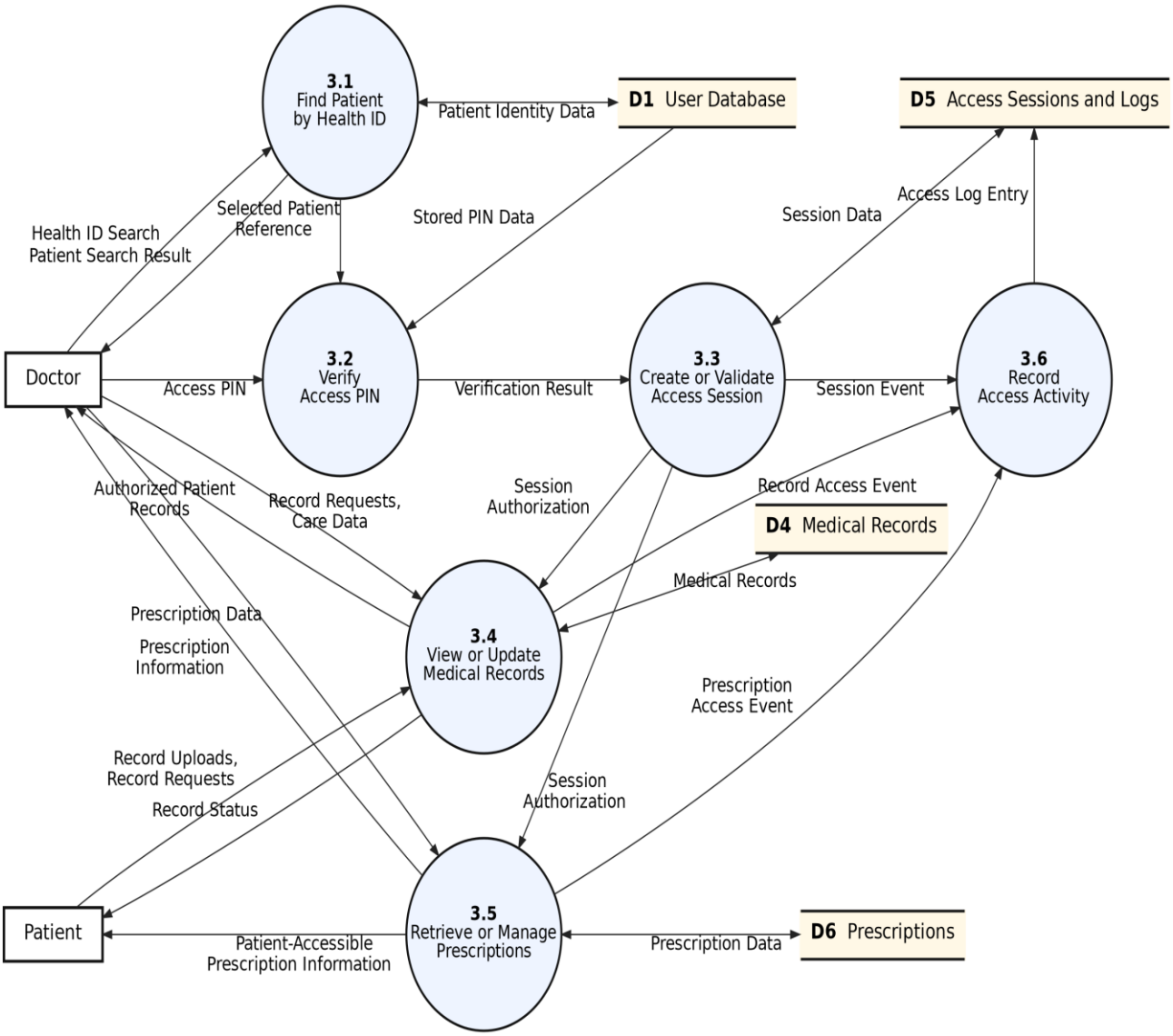


Figure2.6:DFDLevel2–MedicalRecordsandDoctorAccess(Process3.0)

2.2.8 Prescription Module

The prescription module stores prescriptions as structured records rather than relying solely on uploaded documents. A prescription can contain:

- Diagnosis
- Symptoms
- Medicines
- Dosage and frequency
- Duration and timing

Required tests
Medical advice Follow-up
date
Relevant vitals

Doctors can create prescriptions for authorized patients, while patients can retrieve their prescriptions and distinguish active prescriptions from completed ones.

2.2.9 Financial Assistance Module

The financial-assistance subsystem implements the core proposal idea of connecting healthcare costs with financial support. A loan application stores treatment type, estimated treatment cost, requested amount, income, existing EMIs, employment type, credit history, tenure, and the resulting financial evaluation fields. The implementation currently uses two complementary mechanisms.

Rule-Based Eligibility. A deterministic scoring function evaluates the application using factors such as:

Requested amount relative to monthly income
Debt-to-income condition
Employment type Credit
history
A fixed age contribution in the current implementation

The rule score is accumulated using weighted criteria and is also used for EMI calculation.

ML Prediction Service. A separate Flask service exposes a /predict API endpoint. The backend sends application features to this service through Axios. The current repository implementation uses the service as an MVP prediction component. It computes a prediction score from income, credithistory information, and loan amount and returns an approval decision, score, maximum loan amount, interest-rate information, and a reason.

The backend combines the ML response with the rule-based score to obtain a demonstration-level final decision of approved, partial, or rejected. This modular design allows the current prediction component to be replaced by a trained model later without redesigning the main healthcare backend.

Figure 2.7 decomposes Process 4.0 of the Level 1 DFD. The patient submits the loan application details (*4.1*), which are validated (*4.2*). The validated data is used to calculate the rule-based score (*4.3*) and is also passed to *4.4 Request ML Prediction*, which sends a prediction request to the external ML Prediction Service and receives the prediction result. The rule-based score and the prediction result are combined into a single eligibility assessment (*4.5*). Finally, *4.6* stores the application and assessment in the Loan Applications store (D7) and returns the eligibility result to the patient.

The patient never communicates with the prediction service directly; all applicant data passes through the backend. In this workflow the service only produces a prediction and does not train a model. As described above, the current service is an MVP prediction component that can later be replaced by a trained model without changing this data flow.

2.2.10 Fundraising Module

The implementation also contains the proposed funding-gap workflow. A patient can create a fundraising campaign containing:

Campaign title and story

Total treatment requirement
Loan-approved amount Funding
gap
Treatment and hospital information
Supporting medical proof Campaign
deadline

Donations are stored against campaigns and the amount raised and remaining funding gap are updated accordingly. The current implementation represents the payment operation as a completed donation within the MVP rather than implementing a production payment gateway. This leaves payment integration as a future enhancement.

2.2.11 Emergency and NFC-Oriented Workflow

The application contains a public emergency route associated with a patient's Health ID. The intended workflow is for an NFC card/tag to encode an emergency-profile URL.

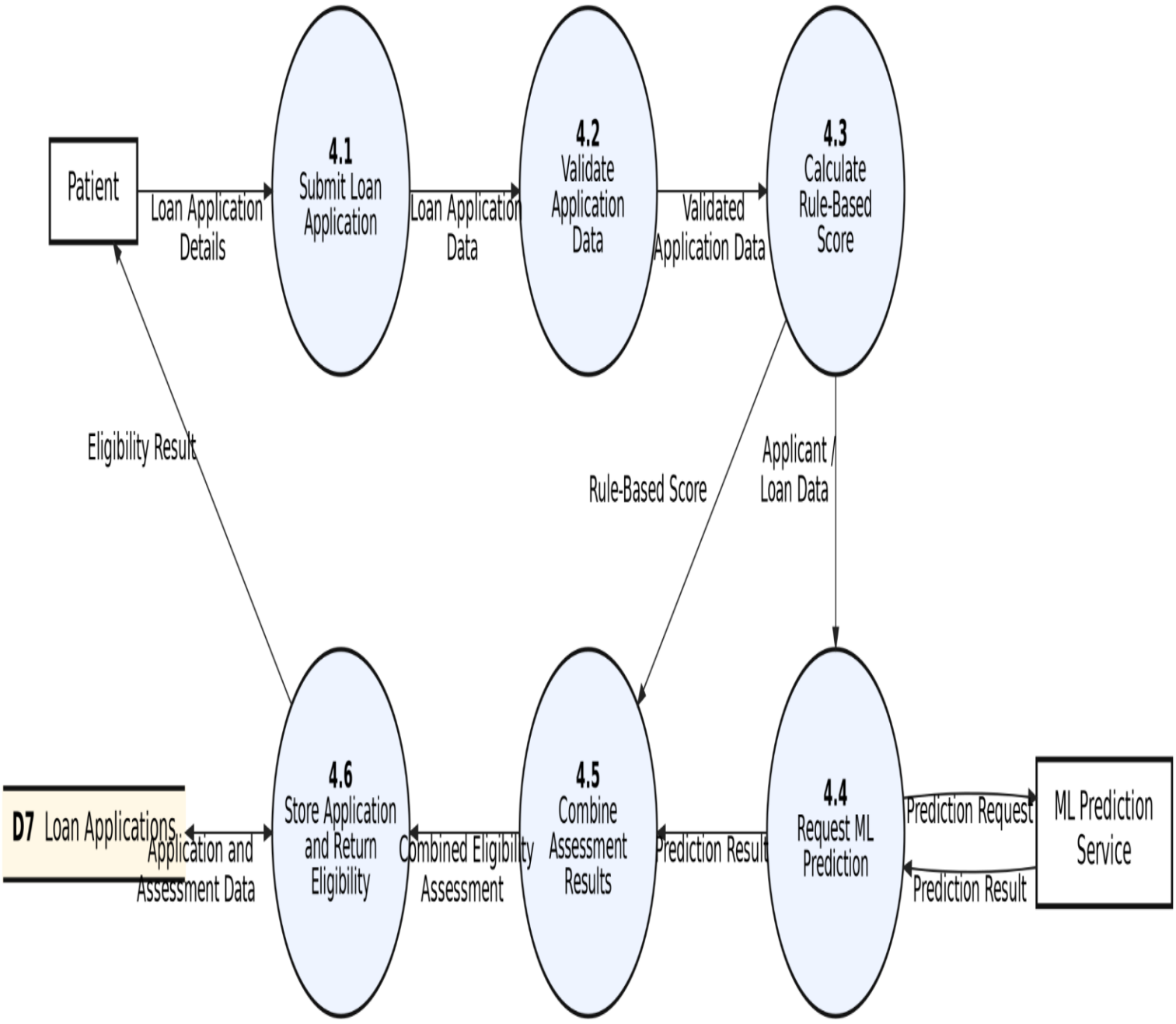
The architecture document specifies NFC write/read operations through the Web NFC API and a public emergency page that exposes limited information without login. The emergency information is intentionally restricted and does not include complete medical records, prescriptions, financial information, or the patient's full personal address.

2.3 Current Implementation Status

The repository at the mid-semester stage contains a functioning full-stack application structure with significant portions of the planned MVP implemented. The implemented scope currently includes:

Patient and doctor registration
Login and token-based authentication
Automatic Health ID generation
Patient profile management
Emergency profile management Health
ID page
Hospital discovery and filtering Hospital
detail pages
Appointment creation and status handling
Medical-record upload and management
Doctor patient search
Temporary PIN-based doctor access
Access auditing and session revocation
Structured prescriptions on the backend
Loan application and eligibility workflow ML-
service API integration
Fundraiser and donation backend workflows
Emergency public-access route

Some frontend routes are still represented by placeholder pages. In particular, portions of the prescription, access-log, financial-result, financial-history, fundraiser-management, and some doctor workflows remain to be integrated fully into the UI.


Patient
Loan Application Details
4.1
Submit Loan Application
Loan Application Data
4.2
Validate Application Data
Validated Application Data
4.3
Calculate Rule-Based Score
Eligibility Result
Rule-Based Score
Applicant / Loan Data
D7 Loan Applications
Application and Assessment Data
4.6
Store Application and Return Eligibility
Combined Eligibility Assessment
4.5
Combine Assessment Results
Prediction Result
4.4
Request ML Prediction
Prediction Request
Prediction Result
ML Prediction Service

Chapter 3

Results and Evaluation

3.1 Testing Methodology

At the present stage, testing has focused primarily on verifying that the implemented frontend, backend, database models, and service interfaces work together correctly.

3.1.1 Functional Testing

The functional testing focus includes:

- Patient registration followed by login
- Doctor registration followed by login
- Protected access to patient and doctor routes
- Patient profile retrieval and update
- Health ID generation and retrieval Hospital search and filtering
- Appointment creation and retrieval
- Medical-record creation and retrieval
- Doctor search for a patient by Health ID

3.1.2 Security-Oriented Testing

Security-related implementation checks include:

- Password hashing using bcrypt
- Access-PIN hashing using bcrypt
- JWT authentication
- Refresh-token validation
- Protected backend routes
- Role-based frontend route guards
- Rate limiting on authentication endpoints Helmet security middleware
- Validation through Joi schemas
- PIN verification and creation of a temporary access session Session revocation
- Prescription creation through doctor-side backend endpoints
- Financial eligibility evaluation
- Communication with the separate prediction API
- Fundraiser creation and donation updates
- Restriction of patient-record queries to the authenticated patient or authorized doctor workflow

---

3.1.3 Integration Testing

Integration testing at this stage centers on communication between:

- React frontend and Express REST API
- Express backend and MongoDB through Mongoose
- Express backend and Cloudinary
- Express backend and the Flask prediction service

3.2 Performance Metrics

The original project proposal defines evaluation criteria such as patient workflow completion time, recommendation usefulness, document access reliability, user satisfaction, and system availability. At the mid-semester stage, the repository does not contain sufficient measured experimental data to claim final numerical performance targets. Therefore, this report records the implementation status instead of inventing benchmark values.

Evaluation Area	Target Stage	Current Status
Authentication	Functional	Implemented for patient and doctor roles
Health ID	Functional	Automatic unique Health ID generation implemented
Hospital discovery	Functional	Search, filtering, sorting and pagination implemented
Medical records	Functional	CRUD, upload and mock hospital-fetch workflow implemented
Doctor access	Functional	PIN verification, 24-hour session and audit logs implemented
Appointments	Functional	Patient request and doctor status workflow implemented
Financial assistance	Prototype	Rule-based scoring and ML-service integration implemented
Emergency access	Prototype	Emergency profile and public Health-ID route implemented
Fundraising	Prototype	Campaign and donation backend workflow implemented

Table 3.1: Mid-Semester Implementation Status

3.3 Analysis

The project has progressed substantially beyond the initial conceptual stage.

3.3.1 Objectives Achieved

The strongest progress has been made in the foundational healthcare platform:

The proposed multi-role authentication system has been implemented.

A centralized patient Health ID has been integrated into the data model and doctor-access workflow.

Hospital and doctor discovery has moved from proposal stage to working backend and frontend modules.

Medical records now have a persistent data model and file-upload workflow.

The privacy objective has been implemented through temporary doctor access and access logging.

Financial assistance is represented through both a rule-based engine and a separate predictionservice interface.

3.3.2 Challenges Encountered

The main challenges at the mid-semester stage are related to integration and completion rather than the basic architecture.

First, the project contains multiple independent workflows that must share authentication and patient identity securely. This increases the need for consistent route protection and permission checks.

Second, healthcare information is sensitive. The design therefore has to distinguish between general profile information, emergency information, and complete medical records.

Third, the financial component combines deterministic rules with a separate ML service. The backend must therefore continue functioning even when the external model service is unavailable.

Finally, some backend functionality has been implemented ahead of the corresponding frontend screens. This is visible in the repository where several application routes currently use placeholder components.

3.3.3 Current Limitations

The following limitations are present in the current mid-semester implementation:

- Several frontend routes are still placeholders.
- The current Flask prediction endpoint is an MVP/demo prediction implementation rather than a fully trained production model.
- Production payment-gateway integration has not yet been implemented for donations.
- The project currently uses a mock hospital data-fetch workflow.
- Formal automated unit and integration test coverage is not yet represented in the repository at the level planned in the architecture specification.
- Final deployment, usability studies, and numerical performance evaluation remain future tasks.

Chapter 4

Conclusion

4.1 Summary of Work

MediBridge was proposed as a centralized healthcare access and financial assistance platform that would simplify healthcare discovery, medical record management, financial-support workflows, and emergency access.

During the period leading to the mid-semester evaluation, the project progressed from the proposal and architecture stage to a substantial full-stack MVP implementation. The developed system now contains a React frontend, Node.js/Express REST backend, MongoDB persistence layer, cloud-based file-storage integration, and a separate Flask prediction service.

The core healthcare workflow has been implemented through user authentication, patient profiles, Health IDs, hospital discovery, appointments, medical records, doctor interaction, privacy-controlled access, and structured prescriptions. Financial assistance, fundraising, and emergency-profile workflows have also been established at the prototype level.

4.2 Key Findings

The three-tier architecture is suitable for the project: separating the React client, Express backend, and MongoDB persistence allows the different healthcare workflows to be developed modularly.

Patient identity is an effective integration point: the MediBridge Health ID connects registration, doctor lookup, patient records, consent management, and the emergency-access workflow.

Temporary authorization improves privacy control: PIN-based access sessions provide a concrete mechanism for granting doctors time-limited access rather than exposing patient records permanently.

Financial assistance benefits from modular design: separating rule-based eligibility logic from the ML prediction API makes it possible to evolve the prediction component independently of the main platform.

The current implementation is ahead of the basic MVP foundation but not yet production-complete: important work remains in frontend integration, automated testing, deployment, real-world integrations, and user evaluation.

4.3 Future Work and Recommendations

The next development stage will focus on completing the remaining functional and integration work.

1. Complete the frontend implementation of prescription viewing, access logs, active access sessions, loan results, loan history, fundraiser management, and remaining doctor workflows.
2. Integrate the existing backend prescription and consent endpoints into their corresponding frontend pages.
3. Replace the current demonstration prediction logic with the intended trained ML model and perform systematic model evaluation.
4. Complete NFC write/read integration and validate the emergency access workflow on supported mobile devices.
5. Add comprehensive unit and integration tests for authentication, access control, records, appointments, and financial workflows.
6. Improve database indexing and query optimization as the healthcare dataset grows.
7. Add production-grade donation/payment processing instead of the current MVP completion logic.
8. Replace the mock hospital-fetch integration with suitable external healthcare interoperability mechanisms where feasible.
9. Conduct usability testing with representative users and measure workflow completion time, document-access reliability, satisfaction, and system availability as defined in the original proposal.
10. Complete cloud deployment and end-to-end validation of the platform.

4.4 Final Remarks

The mid-semester implementation demonstrates that the proposed MediBridge concept can be translated into a modular full-stack software system. The foundational architecture and several of the most important patient and doctor workflows are already implemented.

The remaining development effort is primarily focused on completing frontend integration, strengthening testing and security validation, refining the financial prediction component, and preparing the platform for deployment and final evaluation.